



DATAEXPERT IA

Fundamentos y Algoritmia para Inteligencia Artificial

Informe de análisis

Dataset analizado: Iris

Fecha de generación: 20/09/2026 18:10

1. Introducción

DataExpert IA es una plataforma que aplica fundamentos matemáticos y algorítmicos (álgebra lineal, estadística y Machine Learning) sobre datasets reales o generados sintéticamente, para demostrar de forma práctica el flujo completo de un proyecto de ciencia de datos. Este informe documenta, paso a paso y con los datos reales de la sesión de análisis actual, cada etapa: desde la exploración inicial del dataset hasta el entrenamiento, la evaluación y la comparación de modelos de Machine Learning, pasando por operaciones de álgebra lineal con NumPy y estadística descriptiva —incluyendo funciones de varianza y desviación estándar implementadas manualmente para verificar los resultados de NumPy. Ningún resultado mostrado a continuación es simulado: todos provienen de la ejecución real del pipeline sobre el dataset seleccionado.

El documento sigue el mismo orden en el que normalmente se aborda un proyecto de ciencia de datos: primero se entiende el problema y los datos disponibles (secciones 2 y 3), luego se preparan esos datos para que un algoritmo pueda aprender de ellos (sección 4), se revisan sus propiedades matemáticas y estadísticas (secciones 5 a 9), se visualizan sus patrones (sección 10), se entrena y compara más de un modelo (secciones 11 a 14), se revisan predicciones concretas (sección 15) y, finalmente, se resume todo en una conclusión (sección 16). Cada gráfico incluido va acompañado de un párrafo que interpreta lo que muestra, para que el informe pueda leerse de forma autónoma sin haber usado la plataforma.

2. Dataset seleccionado



Nombre	Iris
Descripción	Clasificación de especies de flores según medidas de sépalos y pétalos.
Variable objetivo	species
Tipo de problema	Clasificación

Al tratarse de un problema de **clasificación**, el objetivo del análisis es predecir la categoría de la variable "**species**" a partir del resto de columnas del dataset.

En un problema de clasificación, el modelo aprende a asignar cada registro a una de un número limitado de categorías (por ejemplo, sí/no, o un conjunto pequeño de clases), en vez de un número continuo. Por eso las métricas de evaluación de las secciones 11 a 14 son accuracy, precision, recall y F1-score en

lugar de un error numérico.

3. Exploración de datos

El dataset contiene **150** registros y **5** variables (4 numérica(s) y 1 categórica(s)). Se detectaron 1 fila(s) duplicada(s), que conviene eliminar antes de entrenar un modelo.

Registros	150
Columnas	5
Variables numéricas	sepal_length, sepal_width, petal_length, petal_width
Variables categóricas	species
Valores nulos totales	0
Filas duplicadas	1

4. Preprocesamiento

El preprocesamiento prepara los datos para que un modelo de Machine Learning pueda entrenarse correctamente: completa los valores que faltaban (con el promedio en columnas numéricas y el valor más común en categóricas), elimina filas repetidas, cambia las categorías de texto por números (porque los modelos solo entienden números) y pone todas las variables numéricas en una misma escala, para que ninguna pese más que otra solo por tener números más grandes.

Métrica	Antes	Después
Filas	150	149
Columnas	5	5
Valores nulos	0	0
Duplicados	1	0

Transformaciones aplicadas en esta sesión:

- Se eliminaron 1 fila(s) duplicada(s).
- Variable objetivo "species": sus categorías de texto se cambiaron por números para poder entrenar el modelo, guardando a qué categoría original corresponde cada número: {0: 'setosa', 1: 'versicolor', 2: 'virginica'}.
- Se pusieron 4 columna(s) numérica(s) en una misma escala común (promedio 0, dispersión 1), para que una variable con números grandes (como un sueldo) no pese más que otra con números chicos (como una edad) solo por su tamaño (técnica: StandardScaler).

5. Álgebra lineal

Estas operaciones son la base matemática de muchos algoritmos de Machine Learning: los vectores representan los datos de un registro o los pesos de un modelo, el producto punto mide su similitud

direccional, la norma mide su magnitud, y resolver un sistema de ecuaciones lineales equivale a encontrar los coeficientes que ajustan un modelo lineal a un conjunto de restricciones. Esta sección es independiente del dataset seleccionado: usa vectores y matrices fijos para ilustrar cada operación de forma clara y verificable.

Operación con vectores	Resultado
Vector v1	[4.00, -2.00, 7.00]
Vector v2	[1.00, 5.00, 3.00]
Suma v1 + v2	[5.00, 3.00, 10.00]
Resta v1 – v2	[3.00, -7.00, 4.00]
Escalar × v1 (× 3)	[12.00, -6.00, 21.00]
Producto punto v1 · v2	15.000
Norma (magnitud) de v1	8.307
Norma (magnitud) de v2	5.916

El producto punto entre v1 y v2 es 15.000: al ser distinto de cero, indica que los vectores apuntan en una dirección similar. La norma mide la magnitud (longitud) de cada vector: v1 tiene una magnitud de 8.307 y v2 de 5.916.

Operación con matrices	Resultado
Matriz A	[[2.00, 1.00], [0.00, 3.00]]
Matriz B	[[1.00, 4.00], [2.00, 1.00]]
Suma A + B	[[3.00, 5.00], [2.00, 4.00]]
Resta A – B	[[1.00, -3.00], [-2.00, 2.00]]
Multiplicación A × B	[[4.00, 9.00], [6.00, 3.00]]
Transpuesta de A	[[2.00, 0.00], [1.00, 3.00]]

Sistema de ecuaciones lineales: $2x + y = 5$, $x + 3y = 6$. Resolviéndolo con *np.linalg.solve* se obtiene la solución $x = 1.800$, $y = 1.400$. La verificación consiste en sustituir esa solución en el sistema original ($A \cdot x = [5.0, 5.999999999999999]$) y comprobar que coincide con el vector b original, confirmando que la solución es correcta.

6-8. Estadística, varianza y desviación estándar

La media y la mediana describen el centro de cada variable; la varianza y la desviación estándar describen qué tan dispersos están los valores alrededor de ese centro. A continuación se muestran ambas medidas de dispersión calculadas de dos formas: con NumPy y con una función propia implementada matemáticamente, para verificar que ambos métodos coinciden.

Variable	Media	Mediana	Var. NumPy	Var. manual	Desv. NumPy	Desv. manual
sepal_length	0.00	-0.05	1.000	1.000	1.000	1.000
sepal_width	0.00	-0.14	1.000	1.000	1.000	1.000
petal_length	-0.00	0.31	1.000	1.000	1.000	1.000
petal_width	-0.00	0.14	1.000	1.000	1.000	1.000
species	0.99	1.00	0.664	0.664	0.815	0.815

La diferencia entre el resultado de NumPy y el de la función manual es cercana a cero en todas las variables, confirmando que la implementación manual (promedio de las diferencias al cuadrado respecto a la media, y su raíz cuadrada) es matemáticamente equivalente a la de NumPy.

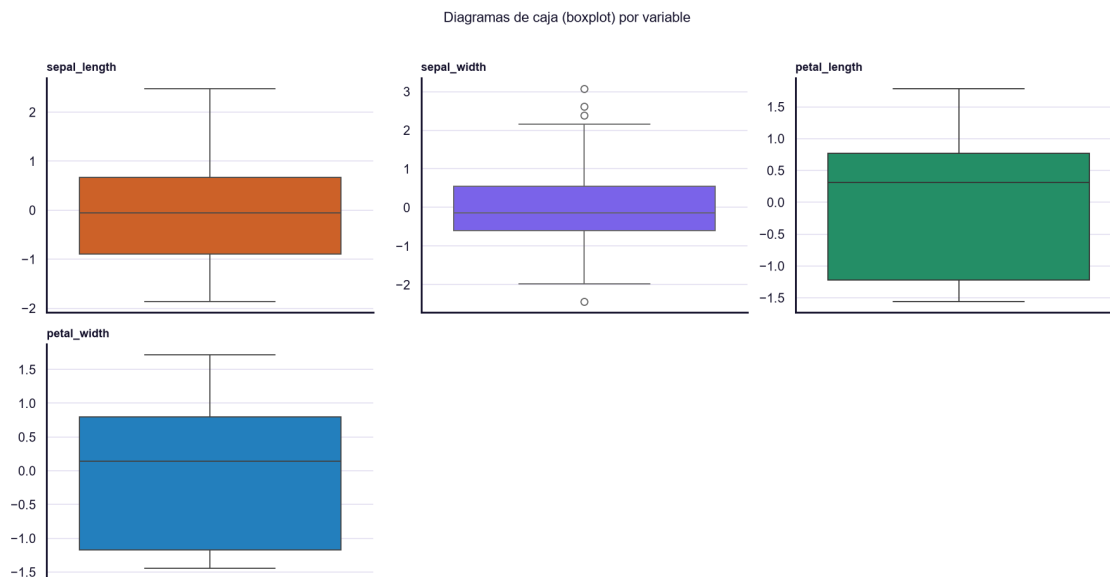
9. Valores atípicos

Se consideran valores atípicos los que se alejan de la media más de $k = 2.0$ desviaciones estándar (fuera del rango $\text{media} \pm k \cdot \sigma$). Este método es sencillo de calcular y asume que los datos siguen aproximadamente una distribución normal.

Variable	Media	Desv. estándar	Límite inf.	Límite sup.	Outliers
sepal_length	0.00	1.00	-2.00	2.00	6
sepal_width	0.00	1.00	-2.00	2.00	5
petal_length	-0.00	1.00	-2.00	2.00	0
petal_width	-0.00	1.00	-2.00	2.00	0
species	0.99	0.82	-0.64	2.62	0

Se detectaron **11** valor(es) atípico(s) en total. La variable con más outliers es "sepal_length", con 6. Estos valores se calculan sobre el dataset ya preprocesado (normalizado): por eso la media de cada variable numérica es cercana a 0 y su desviación estándar cercana a 1 en todas ellas — es el resultado esperado de aplicar StandardScaler, no un error.

Vista visual: diagramas de caja (boxplot)

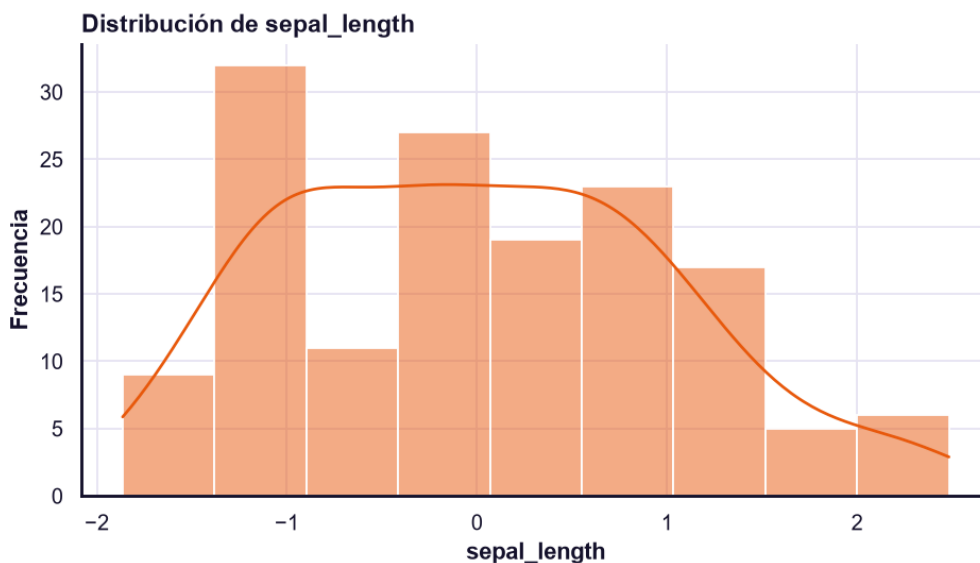


Cada caja te muestra dónde se concentra la mitad de los datos "del medio" de esa variable: la línea dentro de la caja es el valor central (mediana), y los bordes de la caja marcan hasta dónde llega el 25% más bajo y el 25% más alto de esa franja central. Las líneas que salen de la caja (los "bigotes") cubren el resto de los valores normales, y los puntos sueltos por fuera son casos inusuales para esa variable (bastante distintos al resto) — una forma distinta, pero parecida, de detectar casos raros a la de la tabla de valores atípicos de la sección anterior (que usaba el promedio y qué tanto se alejan los datos de él). En este dataset, 1 de 4 variable(s) mostradas tienen al menos un caso así de inusual.

10. Gráficos

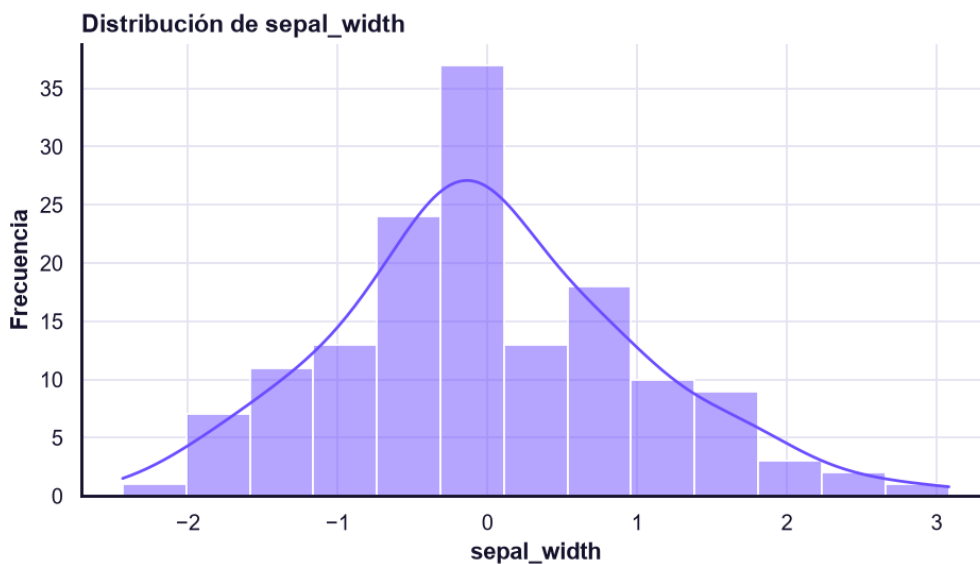
Esta sección presenta, uno por uno (nunca combinados en un solo panel), los gráficos generados con Matplotlib y Seaborn a partir de los datos reales del dataset actual: la forma de cada variable numérica, la relación entre pares de variables, la correlación entre todas ellas y el balance de la variable objetivo. Cada gráfico incluye debajo una breve interpretación calculada a partir de esos mismos datos. Estos valores se calculan sobre el dataset ya preprocesado (normalizado): por eso la media de cada variable numérica es cercana a 0 y su desviación estándar cercana a 1 en todas ellas — es el resultado esperado de aplicar StandardScaler, no un error.

Histograma: distribución de sepal_length



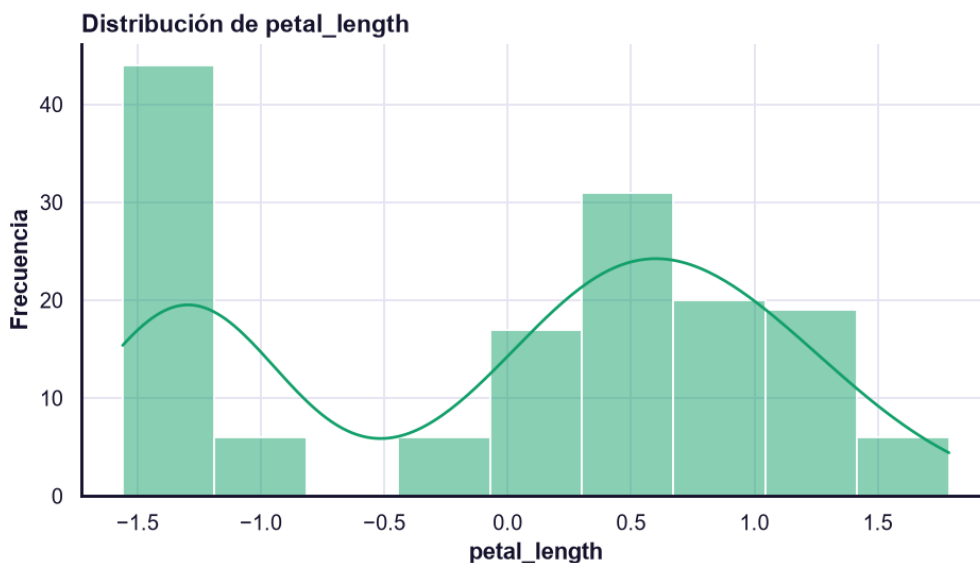
La variable "sepal_length" tiene un promedio de 0.00 sobre 149 datos (la mitad de los casos queda por debajo de -0.05, su valor central). En promedio, cada dato se aleja de ese promedio en unas 1.00 unidades. En cuanto a su forma, sus valores se reparten de forma bastante pareja a los dos lados del centro. El promedio y el valor central están cerca uno del otro, lo cual va de la mano con esa forma pareja.

Histograma: distribución de sepal_width



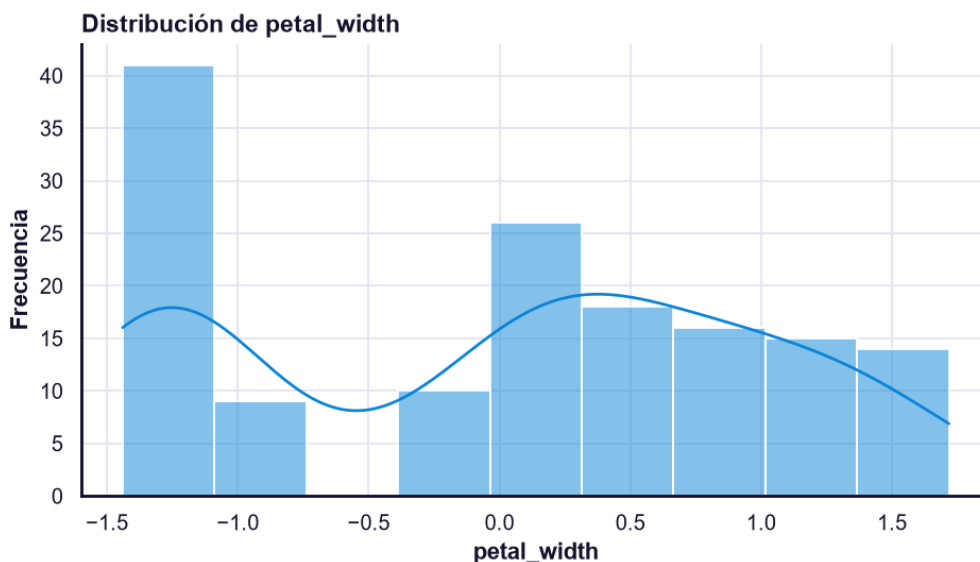
La variable "sepal_width" tiene un promedio de 0.00 sobre 149 datos (la mitad de los casos queda por debajo de -0.14, su valor central). En promedio, cada dato se aleja de ese promedio en unas 1.00 unidades. En cuanto a su forma, sus valores se reparten de forma bastante pareja a los dos lados del centro. El promedio y el valor central están cerca uno del otro, lo cual va de la mano con esa forma pareja.

Histograma: distribución de petal_length



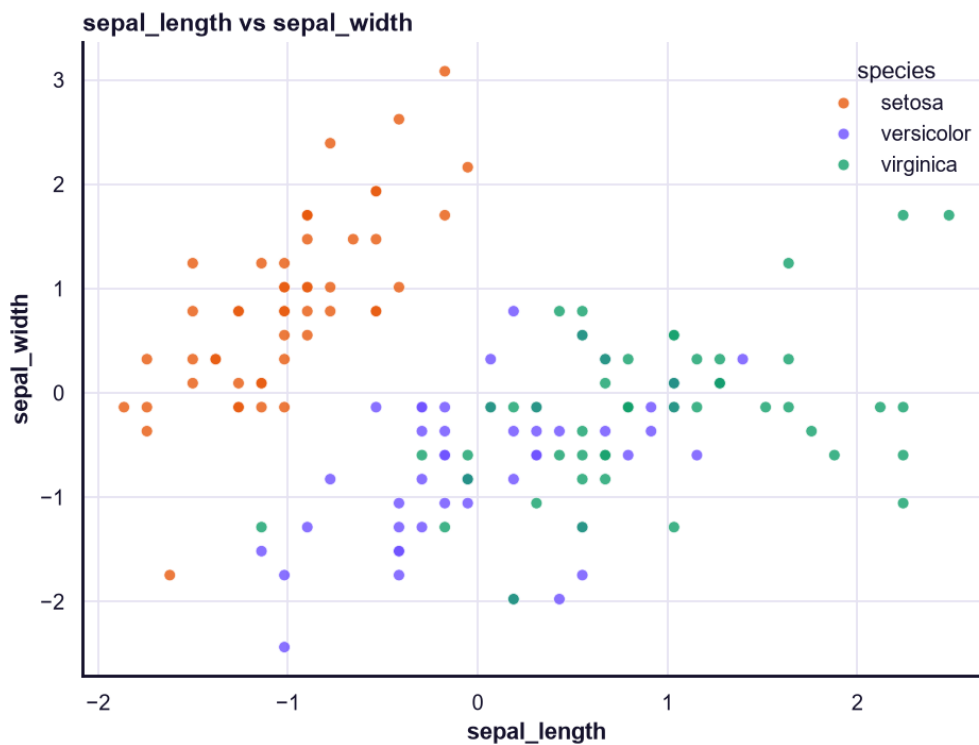
La variable "petal_length" tiene un promedio de -0.00 sobre 149 datos (la mitad de los casos queda por debajo de 0.31, su valor central). En promedio, cada dato se aleja de ese promedio en unas 1.00 unidades. En cuanto a su forma, sus valores se reparten de forma bastante pareja a los dos lados del centro. El promedio y el valor central están cerca uno del otro, lo cual va de la mano con esa forma pareja.

Histograma: distribución de petal_width



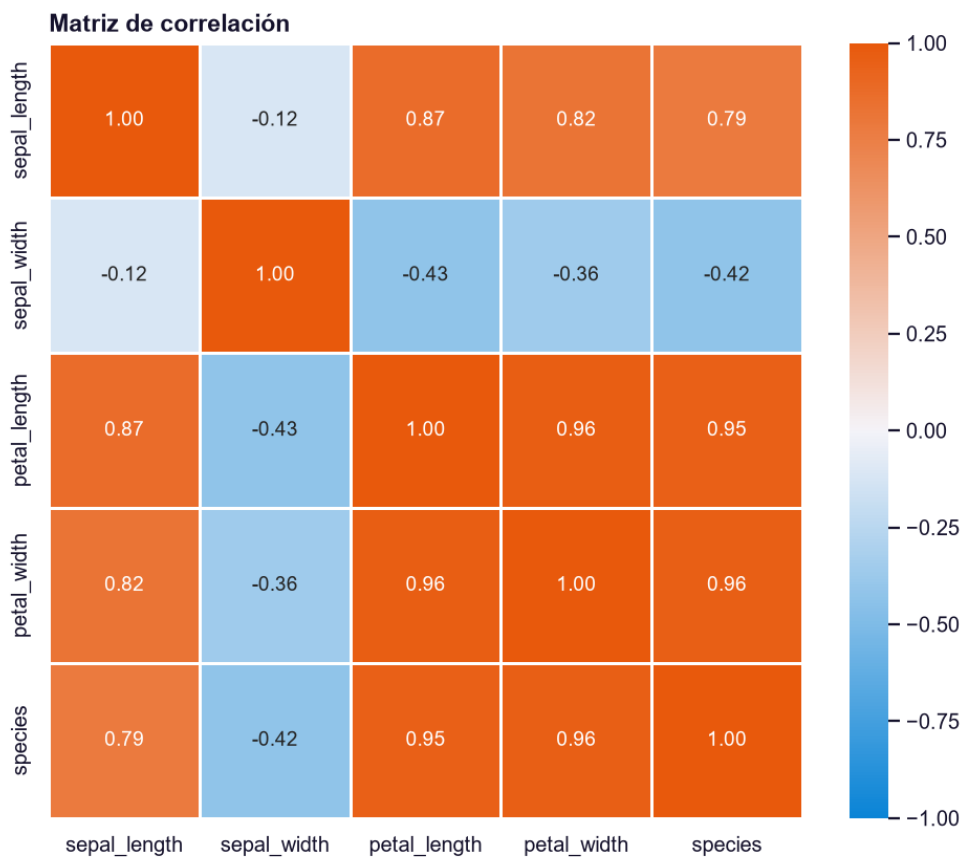
La variable "petal_width" tiene un promedio de -0.00 sobre 149 datos (la mitad de los casos queda por debajo de 0.14, su valor central). En promedio, cada dato se aleja de ese promedio en unas 1.00 unidades. En cuanto a su forma, sus valores se reparten de forma bastante pareja a los dos lados del centro. El promedio y el valor central están cerca uno del otro, lo cual va de la mano con esa forma pareja.

Dispersión: sepal_length vs sepal_width



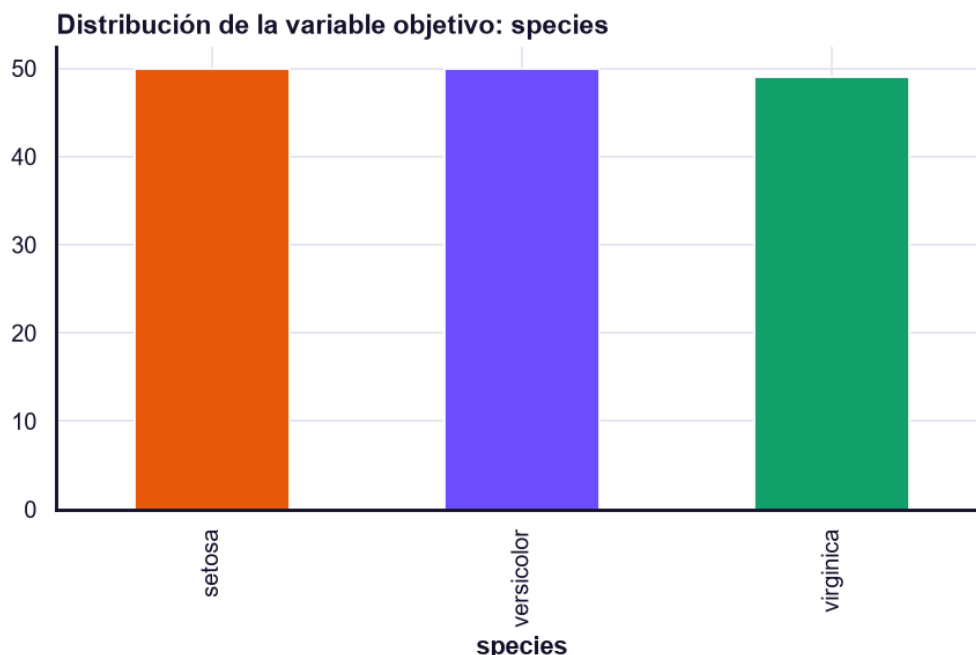
Cada punto es un registro del dataset ubicado según sus valores de "sepal_length" (eje X) y "sepal_width" (eje Y). Si los puntos forman una tendencia clara (una especie de línea), significa que ambas variables están relacionadas; si se ven como una nube sin forma, no lo están. Aquí esa relación (llamada correlación, un número de -1 a 1) es de -0.118: una relación débil y negativa (cuando una crece, la otra tiende a bajar). Los puntos están coloreados según "species" para ver si ese agrupamiento visual coincide con alguna zona del gráfico.

Matriz de correlación



Cada celda mide qué tan relacionadas están dos variables entre sí (correlación), con un número de -1 a 1: cerca de 1 significa que cuando una sube la otra también sube, cerca de -1 significa que cuando una sube la otra baja, y cerca de 0 significa que no hay relación clara entre ellas. Los colores más intensos (rojo o azul fuerte) marcan las relaciones más marcadas. El par con la correlación más fuerte es "petal_length" y "petal_width" (0.963), una relación fuerte y positiva.

Distribución de la variable objetivo: species



La variable objetivo "species" tiene 3 categoría(s) distinta(s). La más frecuente es "setosa" (33.6% de los registros) y la menos frecuente es "virginica" (32.9%). Las clases están relativamente balanceadas entre sí.

11-13. Machine Learning: entrenamiento y evaluación

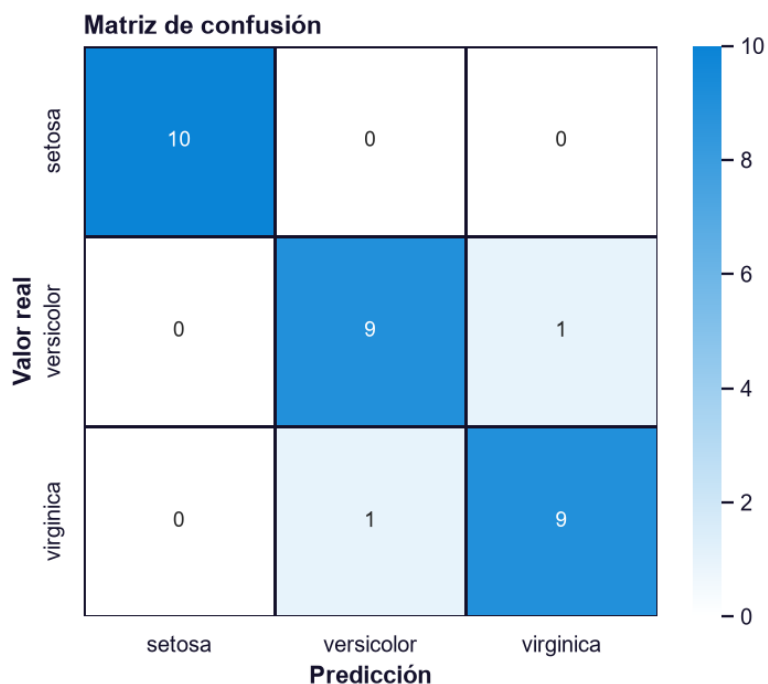
Tipo de problema detectado automáticamente: **Clasificación**. Se separaron los datos en 80% para entrenamiento y 20% para prueba (*train_test_split*, *random_state=42*), y se compararon cuatro modelos de Scikit-learn (KNN, Decision Tree, Random Forest, Logistic Regression) entrenados sobre el mismo conjunto. El de mejor desempeño fue **Decision Tree**.

Sus "hiperparámetros" (ajustes que se eligen antes de entrenar, no algo que el modelo aprenda solo) no se fijaron a mano: se probaron varias configuraciones distintas, cada una validada 10 veces sobre distintas partes de los datos de entrenamiento (nunca los de prueba), y se eligió la que mejor funcionó: **max_depth = 5, min_samples_leaf = 1**.

Accuracy es el porcentaje de predicciones correctas sobre el total de casos evaluados. **Precision** mide qué proporción de los casos que el modelo predijo como positivos lo eran realmente. **Recall** mide qué proporción de los casos positivos reales fue detectada por el modelo. **F1-score** combina precision y recall en un solo valor, especialmente útil cuando las clases no están balanceadas.

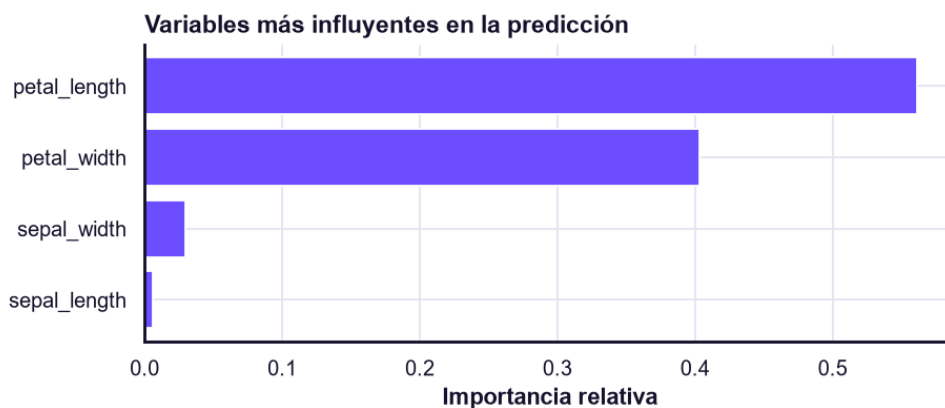
Métrica	Valor
Accuracy	93.33%
Precision	93.33%
Recall	93.33%
F1-score	93.33%

Matriz de confusión



La matriz de confusión compara los valores reales (filas) con las predicciones del modelo (columnas): la diagonal principal son los aciertos, y el resto de las celdas son los errores de clasificación entre cada par de clases.

Variables más influyentes en la predicción



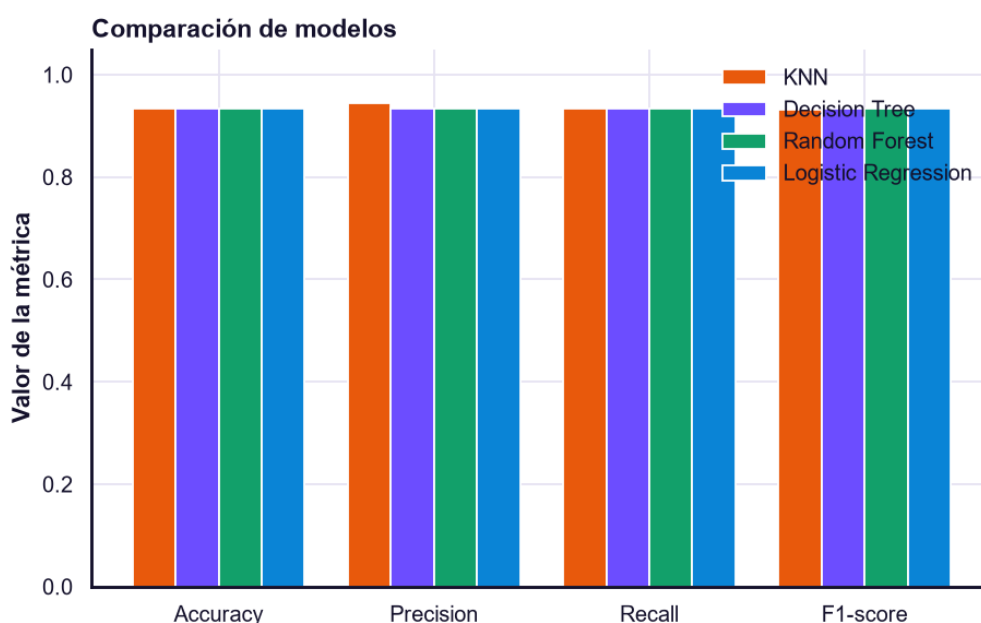
El modelo mide qué tanto usó cada variable para dividir sus decisiones: "petal_length" es la que más influyó en la predicción. Estas importancias siempre suman 1 entre todas las variables del modelo (aquí se muestran solo las más relevantes).

14. Comparación de modelos

Modelo	Accuracy	Precision	Recall	F1
KNN	93.33%	94.44%	93.33%	93.27%
Decision Tree	93.33%	93.33%	93.33%	93.33%
Random Forest	93.33%	93.33%	93.33%	93.33%
Logistic Regression	93.33%	93.33%	93.33%	93.33%

Decision Tree obtuvo el mejor desempeño según F1-score (0.933 frente a 0.933 de Random Forest, una diferencia de 0.000), por lo que fue elegido como el modelo final para este dataset.

Comparación visual



Cada barra representa el valor de esa métrica para un modelo distinto: cuanto más alta, mejor (las 4 métricas mostradas van de 0 a 1). En este dataset, "Decision Tree" tiene las barras más altas en la métrica principal usada para elegir el mejor modelo.

15. Resultados

Se muestran los primeros 12 casos (de 30 evaluados en todo el conjunto de prueba) a modo de muestra. Sobre el conjunto de prueba completo, **28** de 30 casos fueron clasificados correctamente, el mismo 93.3% de accuracy reportado en la sección 11-13.

Valor real	Predicción	Resultado
versicolor	versicolor	Correcto
versicolor	versicolor	Correcto
virginica	virginica	Correcto
setosa	setosa	Correcto
virginica	virginica	Correcto
versicolor	versicolor	Correcto
versicolor	versicolor	Correcto
setosa	setosa	Correcto
virginica	virginica	Correcto
virginica	virginica	Correcto
setosa	setosa	Correcto
setosa	setosa	Correcto

16. Conclusiones

El modelo Decision Tree alcanzó un accuracy de 93.33% sobre el conjunto de prueba, con un F1-score de 93.33%. Estos resultados fueron obtenidos ejecutando el pipeline completo (preprocesamiento, entrenamiento y evaluación) sobre el dataset seleccionado, sin valores simulados. **Decision Tree** obtuvo el mejor desempeño según F1-score (0.933 frente a 0.933 de Random Forest, una diferencia de 0.000), por lo que fue elegido como el modelo final para este dataset.

En cuanto a qué variables explican mejor la predicción, el gráfico de la sección 11-13 muestra que el modelo mide qué tanto usó cada variable para dividir sus decisiones: "petal_length" es la que más influyó en la predicción. Estas importancias siempre suman 1 entre todas las variables del modelo (aquí se muestran solo las más relevantes).

En conjunto, este informe recorrió el flujo completo de un proyecto de ciencia de datos aplicado a Inteligencia Artificial: se exploraron y limpiaron los datos, se revisaron sus propiedades matemáticas y estadísticas con NumPy (incluyendo funciones propias verificadas contra NumPy), se visualizaron sus patrones con Matplotlib y Seaborn, y se entrenó, comparó y evaluó más de un modelo de Scikit-learn sobre el mismo conjunto de datos. Para mejorar estos resultados en un caso real, los siguientes pasos naturales serían probar más combinaciones de hiperparámetros, recolectar más datos (especialmente si alguna clase estaba desbalanceada, ver sección 10), o incorporar variables adicionales relevantes para el problema.